

PHASE 6

# Project Testing Phase

EduGenie - AI Powered Learning Assistant

---

Define and document functional, API, UI, integration, error-handling, and security-oriented tests for EduGenie.

Prepared as a phase-wise academic submission document based on the supplied EduGenie-Ai project archive.

# 1. Test Strategy

Testing should cover the application at three levels: unit tests for task logic and data validation, API tests for FastAPI routes and status codes, and manual UI tests for task selection, request flow, output display, quiz interaction, and copy behavior.

## 2. Test Case Matrix

|       |                                                |                                                     |        |
|-------|------------------------------------------------|-----------------------------------------------------|--------|
| TC-01 | GET /health                                    | Returns status ok and service EduGenie              | High   |
| TC-02 | Load /                                         | Home page loads without template error              | High   |
| TC-03 | Submit empty input                             | Browser blocks request and shows validation message | High   |
| TC-04 | POST /qa with valid text                       | Returns TextResponse result                         | High   |
| TC-05 | POST /explain with topic                       | Returns TextResponse result                         | High   |
| TC-06 | POST /quiz with topic                          | Returns quiz structure containing questions         | High   |
| TC-07 | POST /summarize with passage                   | Returns concise result                              | High   |
| TC-08 | POST /learn/recommendations with topic + level | Returns learning path result                        | High   |
| TC-09 | AI/API exception                               | API returns HTTP 502 and browser shows error status | High   |
| TC-10 | Quiz option selection                          | Correct/incorrect feedback appears                  | Medium |
| TC-11 | Copy result                                    | Visible content copied to clipboard                 | Medium |
| TC-12 | Narrow viewport                                | Layout remains usable below 650px                   | Medium |
| TC-13 | Script output safety                           | Special HTML characters in response are escaped     | High   |
| TC-14 | Missing/invalid secret                         | AI call fails clearly without exposing the key      | High   |

## 3. API Test Examples

```
GET /health -> 200 OK -> {"status":"ok","service":"EduGenie"}
```

```
POST /qa -> JSON body {"text":"What is Python?"} -> 200 OK + result field when the supporting module and AI configuration are present.
```

```
POST /learn/recommendations -> JSON body containing text and level -> 200 OK + learning path response when the supporting module is present.
```

## 4. Negative Testing

- Blank input must not trigger an AI call from the browser.
- Malformed request bodies should be rejected by FastAPI/Pydantic with a client error.
- Task service exceptions should map to 502 as defined in main.py.
- Unexpected or missing quiz data should display a safe fallback message in the UI.

## 5. Current Snapshot Testing Status

A syntax compilation check of main.py succeeds, but the supplied archive cannot be considered fully runnable from the archive contents alone because the modules imported by main.py are missing and templates/index.html is empty. Therefore end-to-end test execution must follow restoration/completion of those files.

## 6. Security Test Checklist

- Verify no API secret appears in source code, screenshots, logs, or public submissions.
- Verify AI response text is escaped before DOM insertion.
- Verify external API failures do not reveal secret configuration values.
- Use a dedicated development credential with appropriate limits and rotate any credential that has been exposed.

## 7. Exit Criteria

The testing phase can be signed off after all high-priority test cases pass, the complete module set imports successfully, the browser UI renders correctly, AI requests return expected structures, error handling works, and screenshots or test logs are archived as evidence.